BulkDB — 대용량 DB 추출 데스크탑 앱 개발 사양서

Oct 23, 2026 · @HHLEE

1. 개요

BulkDB(가칭)는 수백만~수억 행의 쿼리 결과를 GUI 에서 psql `COPY` 와 같은 속도로 CSV/Parquet 파일로 내려받는 Windows·macOS 데스크탑 앱이다. 실행 엔진은 Rust, 화면은 Tauri 위의 Next.js 이며, 데이터는 UI 를 거치지 않는다. 한 줄로 말하면 데이터 표시만 뺀 DB 클라이언트 다. 데이터베이스·스키마·테이블(행 수·용량)·컬럼 구조·인덱스 같은 메타데이터는 모두 보여주되, 테이블의 데이터 자체는 화면에 올리지 않고 파일로만 내보낸다.

배경. 2026-10-23 운영 PostgreSQL 15 에서 1,385만 7,273행(약 3GB)을 추출하는 작업에서 DBeaver Export 는 1시간 넘게 끝나지 않았고 work_mem 설정과 겹쳐 서버 OOM 재시작까지 유발했다. 같은 쿼리를 `psql` + `\copy` 로 SSH 터널 경유 실행하자 **2분 6초**에 끝났다. 차이는 네트워크가 아니라 클라이언트 구조였다. JDBC 기반 GUI 는 행마다 수십 개 객체를 만들었다가 버리지만, COPY 는 서버가 만든 CSV 바이트를 디스크에 그대로 쓴다.

목표.

- 어떤 DB 든 결과 크기와 무관하게 **고정 메모리(100MB 이하)** 로 추출한다.
- PostgreSQL 은 psql `COPY` 와 동일 속도, 그 외 DB 는 JDBC GUI 대비 10배 이상.
- 접속·SSH 터널·세션 설정·출력·진행률·취소를 버튼 몇 개로 끝낸다. 오늘 손으로 한 터널, 인코딩, work_mem, 진행률 확인이 모두 자동화 대상이다.
- 서버를 보호한다. 실행 전 메모리 상한·동시 세션·위험 플랜을 검사해 OOM 같은 사고를 막는다.
- Windows 와 macOS 에서 같은 코드로 같은 동작.
- 드라이버는 DBeaver 처럼 필요할 때 추가하되, 첫 설치는 20MB 이하.

비목표 (v1).

- SQL IDE 기능(자동 완성, 결과 그리드, 데이터 편집)은 만들지 않는다. 쿼리는 사용자가 다른 도구에서 작성해 가져온다.
- 데이터 적재(CSV → DB)는 v1 이후.
- Linux 데스크탑은 구조상 지원되지만 v1 에서 빌드·테스트하지 않는다.
- 테이블 데이터를 화면에 보여주지 않는다. 결과 미리보기도 없다. 메타데이터(행 수·용량·구조·인덱스)까지가 화면의 몫이고, 데이터는 파일의 몫이다.

2. 핵심 설계 원칙

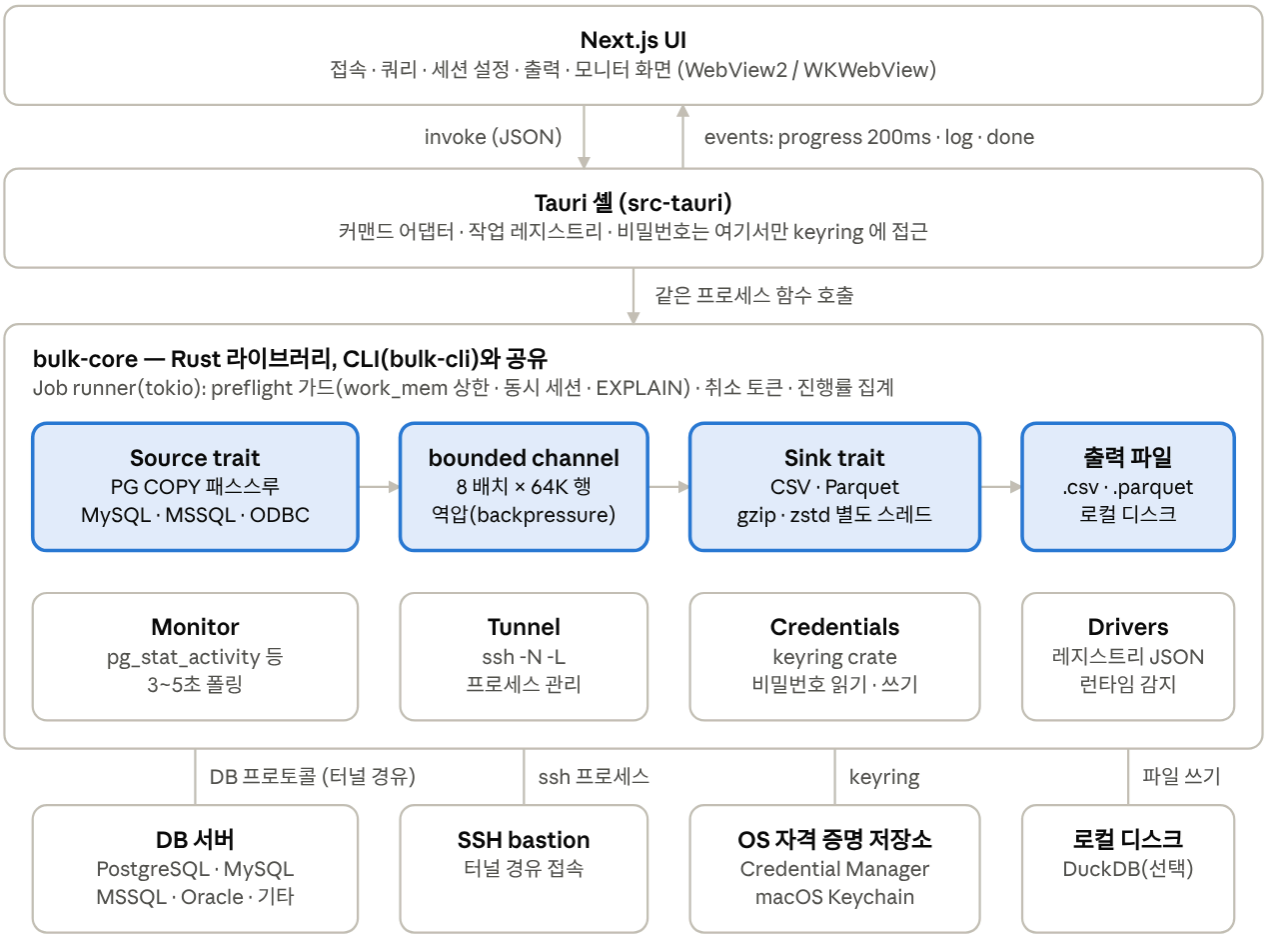
다섯 원칙이 모든 구현 결정보다 앞선다. 코드 리뷰에서 이 원칙에 어긋나는 변경은 되돌린다.

1. **행을 객체로 만들지 않는다.** DB 에서 받은 바이트는 디코딩 없이, 또는 최소 디코딩으로 곧바로 파일 포맷으로 흘러간다. `Vec<Row>` 를 쌓거나 전체 결과를 `collect()` 하는 코드는 금지. PostgreSQL 은 `COPY TO STDOUT` 바이트를 그대로 쓴다.
2. **메모리는 결과 크기와 무관하게 고정.** 읽기와 쓰기 사이는 bounded channel(기본 8 배치 × 64K 행)로 잇는다. 1억 행이든 상주 메모리 100MB 를 넘지 않는 것이 수용 기준이다.
3. **코어와 UI 를 분리.** 모든 로직은 `crates/core` 에 산다. Tauri 셸은 IPC 어댑터, Next.js 는 화면일 뿐이다. 코어는 CLI 로도 똑같이 돌아야 하며, UI 없이 성능을 검증할 수 있어야 한다.
4. **드라이버는 3계층.** 순수 Rust 드라이버(PG·MySQL·MSSQL·SQLite)는 내장, 벤더 클라이언트가 필요한 DB(Oracle·ODBC 전부)는 런타임 감지 후 활성화, DuckDB 같은 부가 모듈은 필요할 때 다운로드. 외부 CLI(psql, bcp)에 실행을 위임하지 않는다.
5. **Windows 와 macOS 는 같은 코드.** 플랫폼 분기는 `platform` 모듈 한 곳에만 둔다(경로, keyring, SSH 바이너리, 벤더 라이브러리 탐색). 그 밖의 코드에 `#[cfg(target_os)]` 가 나타나면 설계 오류다.

그리고 하나 더. **서버를 보호한다.** 실행 전에 서버 메모리 대비 `work_mem` 상한, 같은 테이블을 건드리는 동시 세션, 위험한 실행 계획을 검사해 사용자에게 알린다. 운영 DB 를 OOM 으로 재시작시킨 2026-10-23 사고가 이 원칙의 근거다.

3. 아키텍처

데이터는 UI 를 거치지 않고 Source → Sink → 파일로 흐른다



아키텍처 · 4계층, 데이터 경로는 코어 안에서만

UI 와 Tauri 셸은 명령과 진행률만 주고받고, 데이터는 코어의 Source → bounded channel → Sink 경로로만 흐른다. 모니터는 별도의 가벼운 DB 연결로 세션 상태를 읽고, 터널·자격 증명·드라이버 감지는 코어의 서비스 모듈이 맡는다. 코어는 Tauri 없이 `bulk-cli` 로도 동일하게 실행된다.

프로세스 모델. 앱은 프로세스 하나(Tauri + 코어)에 자식 프로세스 둘(`ssh` 터널, 선택적으로 DuckDB)이다. 추출 작업은 tokio 태스크로 돌고, 작업당 OS 스레드 3개(읽기·변환/쓰기·압축)를 쓴다. 동시 작업 수는 기본 1, 설정으로 최대 3.

4. 기술 스택

Rust 코어, Tauri 2 셸, Next.js 정적 프론트, pnpm + Turborepo 모노레포. 버전은 개발 착수 시점의 안정판으로 고정하고 `rust-toolchain.toml` 과 `package.json` 의 `engines` 로 강제한다.

영역	선택	역할	비고
언어 (코어)	Rust stable, edition 2021	모든 로직	`rust-toolchain.toml` 로 버전 고정
비동기 런타임	tokio	작업 스케줄링, 취소, 채널	Tauri 가 같은 런타임을 씀
데스크탑 셸	Tauri 2	창, IPC, 번들, 업데이트	Windows WebView2, macOS WKWebView
프론트	Next.js 16.4.2 (App Router, `output: 'export'`) + React 19.3.1 + TypeScript 7.0.2	화면	SSR·API Route 사용 금지
UI 컴포넌트	shadcn/ui + Tailwind	폼, 테이블, 로그 패널	데스크탑 밀도에 맞게 간격 축소
상태 관리	zustand	접속 목록, 실행 상태	서버 상태는 Rust 가 단일 진실
패키지 관리	pnpm 12.14.0 + Turborepo 2	JS 워크스페이스, 태스크	Cargo 는 별도 워크스페이스
PostgreSQL	`tokio-postgres` (`copy_out`)	COPY 바이트 스트림	가장 빠른 경로, 항상 내장
MySQL / MariaDB	`mysql_async`	텍스트 프로토콜 스트리밍	내장
MSSQL	`tiberius`	TDS 스트리밍	내장
SQLite	`rusqlite`	로컬 파일 DB	내장, 테스트용으로도 사용
ODBC (그 외 전부)	`odbc-api`	블록 커서 fetch	벤더 드라이버는 사용자 설치
Oracle	`oracle` (ODPI-C)	Instant Client 런타임 로드	2계층 드라이버
CSV 쓰기	`csv`	RFC 4180 인용	수백 MB/s
Parquet 쓰기	`arrow` + `parquet`	컬럼 포맷, ZSTD 압축	별도 스레드
압축	`flate2` (gzip), `zstd`	출력 압축	별도 스레드
자격 증명	`keyring`	OS 저장소 접근	Credential Manager / Keychain

영역	선택	역할	비고
설정 저장	`serde` + `toml`	접속·설정 파일	플랫폼별 config 디렉터리
로깅	`tracing` + 파일 appender	진단 로그	비밀번호 마스킹 필수
에러	`thiserror` (코어), `anyhow` (CLI)	에러 타입	IPC 로는 코드 + 메시지로 직렬화
로컬 분석 (선택)	DuckDB (`duckdb` crate, bundled 아님)	CSV/Parquet 즉시 조회	필요 시 다운로드, 3계층
테스트 DB	Docker (postgres, mysql, mssql)	통합 테스트	CI 에서 서비스 컨테이너

버전 정책. 프론트는 pnpm 12.14.0, Next.js 16.4.2, React 19.3.1, TypeScript 7.0.2 로 고정한다. package.json 의 packageManager 와 engines, .npmrc 의 engine-strict=true 로 강제하고, 다른 버전의 pnpm 으로는 install 이 실패해야 한다. 네 개를 올릴 때는 PR 하나로 함께 올린다. Rust 는 6개월마다 최신 stable 로 올린다. Tauri·Next 는 메이저 업그레이드 시 전체 회귀 테스트를 통과해야 머지한다. 드라이버 크레이트는 보안 패치 외에는 분기 단위로만 올린다.

5. 저장소 구조와 빌드

저장소 하나에 Cargo 워크스페이스(Rust)와 pnpm 워크스페이스(JS)가 공존하고, Turborepo 가 두 쪽의 태스크를 지휘한다. 로직은 `crates/core` 한 곳에만 둔다.

```
bulkdb/
├─ apps/
│  └─ desktop/                  # Tauri 앱 (pnpm 패키지 "desktop")
│     ├─ package.json
│     ├─ next.config.js         # output: 'export'
│     ├─ src/app/               # Next.js App Router 화면
│     ├─ src/lib/ipc.ts         # invoke/listen 래퍼, 타입은 Rust 에서 생성
│     └─ src-tauri/             # Tauri Rust 셸 (얇게 유지)
│        ├─ Cargo.toml          #   bulk-core 를 path 의존성으로
│        ├─ tauri.conf.json
│        ├─ capabilities/       #   Tauri 2 권한 선언
│        └─ src/
│           ├─ main.rs
│           ├─ commands.rs      #   invoke 핸들러만
│           └─ events.rs        #   진행률 emit (200ms 스로틀)
├─ crates/
│  ├─ core/                     # bulk-core: 모든 로직
│  │  └─ src/
│  │     ├─ lib.rs
│  │     ├─ job.rs              #   JobRunner, CancellationToken, Progress
```

```
|  |     ├─ source/              #   trait + postgres.rs, mysql.rs, mssql.rs,
odbc.rs, sqlite.rs
|  |     ├─ sink/                #   trait + csv.rs, parquet.rs, compress.rs
|  |     ├─ monitor/             #   trait + postgres.rs, mysql.rs, mssql.rs
|  |     ├─ preflight.rs         #   work_mem 상한, 동시 세션, EXPLAIN 검사
|  |     ├─ tunnel.rs            #   ssh 프로세스 관리
|  |     ├─ drivers/             #   registry.json 로더, 런타임 감지
|  |     ├─ credentials.rs       #   keyring
|  |     ├─ config.rs            #   접속·설정 파일
|  |     └─ platform/            #   mod.rs + windows.rs + macos.rs (유일한 cfg
분기)
|  └─ cli/                       # bulk-cli: 헤드리스 실행기, Phase 1 검증 도구
├─ packages/
|  └─ ipc-types/                 # Rust 에서 생성한 TS 타입 (ts-rs 또는 specta)
├─ drivers/registry.json         # 드라이버 정의 (7장)
├─ Cargo.toml                    # [workspace] members = ["crates/*",
"apps/desktop/src-tauri"]
├─ rust-toolchain.toml
├─ pnpm-workspace.yaml           # packages: ["apps/*", "packages/*",
"crates/*"]
├─ turbo.json
└─ package.json
```

규칙.

- `src-tauri` 에는 비즈니스 로직을 두지 않는다. `commands.rs` 의 함수는 인자를 코어 타입으로 바꿔 코어를 호출하고 결과를 돌려주는 10줄 안팎이어야 한다.
- `crates/core` 는 Tauri 에 의존하지 않는다. `tauri::State` 같은 타입이 코어에 나타나면 설계 오류.
- IPC 페이로드 타입은 Rust 에 정의하고 `ts-rs` 로 `packages/ipc-types` 에 생성한다. TS 쪽에서 손으로 타입을 쓰지 않는다.
- `platform/` 바깥의 `#[cfg(target_os = …)]` 는 금지.

Turborepo 와 Cargo 의 관계. Turbo 는 `package.json` 스크립트만 실행한다. Rust 크레이트에는 `cargo` 명령을 감싼 얇은 `package.json` 을 두고, Cargo 의 증분 캐시가 따로 있으니 Turbo 캐시는 끈다.

태스크	명령	비고
`pnpm dev`	`turbo run dev` → `tauri dev` (Next dev 서버 자동 기동)	핫 리로드
`pnpm build`	`turbo run build` → Next export → `tauri build`	설치 파일 생성

태스크	명령	비고
`pnpm core:test`	`cargo test -p bulk-core`	단위 + 통합(docker 필요 시 `--features it`)
`pnpm cli -- …`	`cargo run -p bulk-cli -- …`	헤드리스 추출
`pnpm typegen`	`cargo test -p bulk-core --features ts-export`	TS 타입 재생성
`pnpm lint`	`cargo clippy -- -D warnings` + `eslint`	CI 게이트

설정 파일 본문은 부록(17장)에 있다.

6. 코어(Rust) 설계

코어는 네 개의 trait(Source, Sink, Monitor, Catalog)과 그것을 잇는 JobRunner 로 이루어진다. 드라이버를 추가한다는 것은 Source·Monitor·Catalog 구현체 하나씩을 더하는 일이고, 그 밖의 코드는 건드리지 않는다.

6.1 Source — 읽기

```
#[async_trait]
pub trait Source: Send + Sync {
    /// 접속 확인 + 서버 정보(버전, 메모리 설정 등). 접속 테스트와 preflight 가 함께 쓴다.
    async fn probe(&self) -> Result<ServerInfo>;
    /// 쿼리를 실행해 배치 스트림을 돌려준다. 호출 즉시 반환하고, 행은 스트림을 당길 때 흐른다.
    async fn stream(&self, q: &Query, opts: &SessionOpts, cancel: CancellationToken)
        -> Result<BoxStream<'static, Result<Batch>>>;
    /// 가능하면 실행 계획. 없으면 None.
    async fn explain(&self, q: &Query) -> Result<Option<Plan>>;
}

pub enum Batch {
    /// 서버가 이미 CSV 로 만든 바이트. PostgreSQL COPY 전용. 디코딩 없이 Sink 로 간다.
    RawCsv(Bytes),
    /// 컬럼 단위 Arrow RecordBatch. MySQL·MSSQL·ODBC 가 만든다. Parquet 로 바로 쓸 수 있다.
    Arrow(RecordBatch),
}
```

- `SessionOpts` 는 `work_mem`, `statement_timeout`, `lock_timeout`, 클라이언트 인코딩(UTF-8 고정), fetch 배치 크기를 담는다. Source 는 자기 DB 에 맞는 `SET` 으로 번역한다. MySQL 은 `max_execution_time`, MSSQL 은 `SET LOCK_TIMEOUT` 등.
- `RawCsv` 경로는 PostgreSQL 전용이고 출력이 CSV 일 때만 쓴다. Parquet 출력이면 PostgreSQL 도 `COPY … (FORMAT binary)` 를 디코딩해 `Arrow` 로 간다.
- 배치 크기는 64K 행 또는 8MB 중 먼저 차는 쪽. 행이 아주 넓은 테이블에서 메모리가 튀지 않게 바이트 상한을 둔다.

6.2 Sink — 쓰기

```
pub trait Sink: Send {
    fn write(&mut self, batch: Batch) -> Result<()>;
    fn finish(self: Box<Self>) -> Result<SinkReport>;   // 행 수, 바이트, 파일 목록
}
```

- `CsvSink`: `RawCsv` 는 그대로 쓰고, `Arrow` 는 `csv` 크레이트로 인용 처리해 쓴다. 옵션: 헤더, 구분자, BOM, NULL 표기.
- `ParquetSink`: `Arrow` 만 받는다. 행 그룹 128MB, ZSTD.
- `CompressSink<S>`: 다른 Sink 를 감싸 gzip/zstd 로 압축. 별도 스레드에서 돌고 내부적으로 또 하나의 bounded channel 을 둔다.
- `SplitSink<S>`: N 행 또는 M 바이트마다 파일을 나눈다. Excel 용 100만 행 분할이 여기서 나온다.

6.3 JobRunner — 파이프라인

```
Source.stream() ──▶ mpsc::channel(8) ──▶ Sink.write() ──▶ (압축 스레드) ──▶ 파일
     읽기 태스크            backpressure          쓰기 스레드
```

- 채널은 `tokio::sync::mpsc` 용량 8. Sink 가 느리면 Source 가 자연히 멈춘다(역압). 이 숫자가 메모리 상한을 결정한다. 8 × 8MB = 64MB.
- 취소는 `CancellationToken` 하나를 읽기 태스크와 쓰기 스레드가 공유한다. 취소 시 Source 는 DB 세션을 `cancel` 하고(PostgreSQL 은 `pg_cancel_backend` 가 아니라 프로토콜 레벨 Cancel), Sink 는 쓰던 파일을 닫은 뒤 `.partial` 로 이름을 바꾼다.
- 진행률은 `Arc<Progress>` 의 원자 카운터(행, 바이트, 배치 수)로 쌓고, 호출자가 원하는 주기로 읽는다. 코어는 이벤트를 밀지 않는다.
- 에러는 `thiserror` 열거형 `CoreError { Connect, Auth, Query{code, msg}, Io, Cancelled, Timeout, Driver{id, detail} }`. IPC 로는 `{code, message, detail?}` 로 직렬화한다.

6.4 Monitor — 서버 쪽 세션 상태

```
#[async_trait]
pub trait Monitor: Send + Sync {
    async fn snapshot(&self, job: &JobHandle) -> Result<ServerSnapshot>;   // state, wait, elapsed, server_rows?
    async fn active_sessions(&self, touching: &[TableRef]) -> Result<Vec<SessionInfo>>;
}
```

별도의 가벼운 연결로 3~5초마다 호출한다. PostgreSQL 은 `pg_stat_activity` + `pg_stat_progress_copy`, MySQL 은 `performance_schema.threads`, MSSQL 은 `sys.dm_exec_requests` (`percent_complete`, `wait_type`). 작업 세션은 `application_name` 에 `bulkdb-<job id>` 를 넣어 찾는다.

6.5 Catalog — 메타데이터 브라우저

```
#[async_trait]
pub trait Catalog: Send + Sync {
    async fn databases(&self) -> Result<Vec<DatabaseInfo>>;
// 이름, 용량
    async fn schemas(&self, db: &str) -> Result<Vec<SchemaInfo>>;
    async fn tables(&self, db: &str, schema: &str) -> Result<Vec<TableInfo>>;
// 이름, 종류, 추정 행 수, 용량
    async fn columns(&self, t: &TableRef) -> Result<Vec<ColumnInfo>>;
// 이름, 타입, null, 기본값, PK 여부
    async fn indexes(&self, t: &TableRef) -> Result<Vec<IndexInfo>>;
// 이름, 컬럼, unique, 종류, 조건(partial), 용량
    async fn exact_count(&self, t: &TableRef, cancel: CancellationToken) -> Result<u64>;  // 사용자가 요청할 때만
}
```

- 행 수는 기본이 **통계 기반 추정치**다. PostgreSQL `pg_class.reltuples`, MySQL `information_schema.tables.table_rows`, MSSQL `sys.dm_db_partition_stats`. 1,386만 행 테이블에서 `count(*)` 를 자동으로 돌리면 그게 또 하나의 대량 쿼리가 된다. 정확한 값은 사용자가 버튼을 눌렀을 때만, 취소 가능하게.
- 용량은 테이블·인덱스·합계를 따로 돌려준다. PostgreSQL `pg_table_size` / `pg_indexes_size`.
- 인덱스에는 partial 조건(`WHERE …`)과 INCLUDE 컬럼을 그대로 보여준다. 2026-10-23 의 `distribution_id IS NOT NULL` 인덱스처럼, 조건이 보여야 왜 쿼리가 인덱스를 못 타는지 알 수 있다.
- 결과는 접속별로 메모리 캐시하고, 새로고침 버튼으로 갱신한다. 테이블 수천 개 스키마도 한 번의 `information_schema` 조회로 받도록 N+1 쿼리를 금지한다.

6.6 Preflight — 실행 전 검사

실행 버튼을 누르면 Source.probe 와 Monitor, Catalog 를 묶어 아래를 검사하고, 경고가 있으면 실행 전에 보여준다. 차단이 아니라 경고다. 사용자가 확인하면 진행한다.

검사	기준	근거
work_mem 상한	서버 `shared_buffers` 로 RAM 추정(×4). `work_mem` × 3 > RAM 의 10% 면 경고	2026-10-23 512MB 설정으로 4GB 서버 OOM
동시 세션	같은 테이블을 읽는 `active` 세션이 있으면 경고	같은 쿼리 두 개가 겹쳐 메모리 두 배
실행 계획	`Nested Loop` 안에 서브쿼리·Aggregate 가 있으면 "LATERAL 패턴, 행 수 × 반복" 경고	1,386만 회 반복으로 수 시간
결과 크기	계획의 예상 행 수 × 폭으로 파일 크기 추정, 디스크 여유와 비교	3GB 파일
출력 경로	덮어쓰기 여부, 쓰기 권한	

6.7 CLI

`bulk-cli` 는 코어를 그대로 노출한다. Phase 1 의 성능 검증 도구이자, 스케줄러에서 돌릴 때의 인터페이스다.

```
bulk-cli export --conn prod --sql query.sql --out ./out/export.csv --work-mem
64MB --progress
bulk-cli catalog tables --conn prod --schema public
bulk-cli probe --conn prod
```

7. 드라이버 전략

드라이버는 세 계층으로 나뉘고, 첫 설치에는 1계층만 들어간다. 사용자가 "드라이버를 설치한다" 는 것은 2계층의 벤더 클라이언트나 3계층의 선택 모듈을 받는 일이다.

계층	대상	앱에 포함되는 것	사용자가 설치하는 것	앱 크기 영향
1 내장	PostgreSQL, MySQL/MariaDB, MSSQL, SQLite	순수 Rust 드라이버 전체	없음	각 1~3MB
2 벤더 클라이언트	Oracle, DB2, Informix, SAP HANA, Snowflake, 그 외 ODBC 전부	얇은 글루 (수백 KB)	벤더 클라이언트 또는 ODBC 드라이버	거의 0

계층	대상	앱에 포함되는 것	사용자가 설치하는 것	앱 크기 영향
3 선택 모듈	DuckDB(로컬 분석), 커뮤니티 플러그인	없음	앱 안에서 다운로드	0 (받은 만큼)

1계층을 나중 설치로 빼지 않는 이유. 순수 Rust 드라이버는 합쳐도 10MB 가 안 된다. jar 를 수십 MB 씩 받는 JDBC 와 달리 빼서 얻는 게 없고, 넣어두면 "설치" 단계 없이 바로 접속된다.

PostgreSQL 은 예외적으로 특별 취급한다. `COPY TO STDOUT` 바이트 패스스루는 행 디코딩 자체를 건너뛰어 어떤 경로보다 빠르다. ODBC 로도 PostgreSQL 에 붙을 수 있지만 그 경로는 쓰지 않는다.

7.1 드라이버 레지스트리

드라이버 정의는 `drivers/registry.json` 한 파일이다. 앱은 시작할 때 이 파일을 읽어 접속 유형 목록을 만들고, 2계층은 벤더 라이브러리를 **try-load** 해 활성/비활성을 정한다.

```
{
  "id": "oracle",
  "name": "Oracle Database",
  "tier": "vendor-client",
  "source": "oracle",
  "requires": {
    "windows": { "library": "oci.dll",       "probe": ["%ORACLE_HOME%\\bin",
"C:\\oracle\\instantclient*", "%PATH%"] },
    "macos":   { "library": "libclntsh.dylib", "probe": ["$ORACLE_HOME/lib",
"/opt/oracle/instantclient*", "$DYLD_LIBRARY_PATH"] }
  },
  "install": {
    "label": "Oracle Instant Client 21 (Basic)",
    "windows": "https://www.oracle.com/database/technologies/instant-
client/winx64-64-downloads.html",
    "macos":   "https://www.oracle.com/database/technologies/instant-
client/macos-arm64-downloads.html"
  },
  "fields": ["host", "port", "service_name", "user", "password"],
  "defaults": { "port": 1521 },
  "session_opts": ["statement_timeout"]
}
```

필드	의미
`tier`	`builtin` / `vendor-client` / `odbc` / `plugin`

필드	의미
`source`	코어의 Source 구현체 id. `odbc` 면 `odbc-api` 공용 구현을 쓴다
`requires`	플랫폼별로 필요한 라이브러리 파일명과 탐색 경로. `builtin` 은 없음
`install`	미설치 시 보여줄 다운로드 링크. 앱이 대신 설치하지는 않는다(벤더 라이선스)
`fields`	접속 화면에 보일 입력 항목. 공통 항목 외에 DB 별 항목 (`service_name` , `instance` , `dsn`)
`session_opts`	이 DB 가 지원하는 세션 설정 키. 지원 안 하는 항목은 화면에서 숨긴다

7.2 런타임 감지

- 시작 시 2계층 각 항목의 `requires.<platform>.library` 를 `libloading` 으로 열어본다. 성공하면 활성, 실패하면 회색 + "설치 필요" 표시와 `install` 링크.
- ODBC 는 `odbc-api` 로 설치된 드라이버 이름 목록을 읽어, 사용자가 뭘 깔았는지 접속 화면에서 바로 고른다. Windows 는 ODBC 드라이버 매니저가 기본 내장이고, macOS 는 unixODBC(`brew install unixodbc`) 가 필요하다. 없으면 ODBC 계열 전체가 비활성.
- "다시 감지" 버튼은 재시작 없이 재탐색한다.

7.3 DB 별 경로 요약

DB	Source	Monitor	Catalog 원천	행 수 추정	비고
PostgreSQL	`tokio-postgres` COPY	`pg_stat_activity` , `pg_stat_progress_copy`	`pg_catalog`	`reltuples`	CSV 는 바이트 패스스루
MySQL/MariaDB	`mysql_async` 스트리밍	`performance_schema.threads`	`information_schema`	`table_rows`	`max_execution_time` 으로 타임아웃
MSSQL	`tiberius`	`sys.dm_exec_requests`	`sys.*` 카탈로그 뷰	`dm_db_partition_stats`	`percent_complete` 를 진행률 보조로
SQLite	`rusqlite`	없음	`sqlite_master`	`count(*)` 즉시	로컬 파일
Oracle	`oracle` (ODPI-C)	`v$session`	`ALL_TABLES` 등	`NUM_ROWS`	Instant Client 필요
그 외	`odbc-api` 블록 커서	없음 또는 벤더별	`SQLTables` / `SQLColumns`	없음	드라이버 품질에 좌우

7.4 3계층 플러그인 (v2 이후)

커뮤니티 드라이버나 특수 DB 는 DLL 로딩이 아니라 **독립 실행 파일 + stdio 로 Arrow IPC 스트림** 으로 받는다. 크래시가 앱을 죽이지 않고, 어떤 언어로든 만들 수 있고, Arrow IPC 는 파이프를 거쳐도 복사 비용이 거의 없다. v1 에서는 `tier: "plugin"` 을 레지스트리 스키마에만 예약해 두고 구현하지 않는다.

8. 기능 명세

기능은 열 묶음이다. 데이터 표시만 없고, 그 밖에 대량 추출에 필요한 것은 모두 있다. 각 항목의 "완료 기준" 이 QA 의 체크리스트다.

8.1 접속 관리

항목	명세
접속 저장	이름, 드라이버, host, port, database, user, 드라이버별 추가 필드(7.1 `fields`), SSH 터널 설정, 기본 세션 설정, 기본 출력 폴더
비밀번호	OS 자격 증명 저장소에만. 설정 파일에는 참조 키만 남는다. "저장 안 함" 선택 시 실행마다 입력
접속 테스트	터널 기동 → 접속 → `probe` → 서버 버전, RAM 추정, `work_mem` 류 기본값 표시. 실패 시 단계별 원인(터널/인증/네트워크/권한)
가져오기	DBeaver 의 `data-sources.json` 에서 접속 목록 가져오기(비밀번호 제외)
완료 기준	저장 → 재시작 → 테스트 성공. 설정 파일에 비밀번호 문자열이 없음

8.2 SSH 터널

항목	명세
인증	비밀번호, 키 파일(OpenSSH 형식), 에이전트. `.ppk` 는 변환 안내만
실행	OS 의 `ssh` 바이너리를 자식 프로세스로: `ssh -N -C -o ServerAliveInterval=60 -o ExitOnForwardFailure=yes -L 127.0.0.1:<임의 포트>:<db host>:<db port> user@bastion`

항목	명세
포트	로컬 포트는 앱이 빈 포트를 골라 쓴다. 사용자는 보지 않아도 된다
압축	`-C` 기본 켬. 텍스트 전송이라 3~5배 효과
host key	처음 보는 호스트는 지문을 보여주고 확인받는다. `StrictHostKeyChecking=no` 금지
수명	접속 테스트·카탈로그·추출·모니터가 같은 터널을 공유. 마지막 사용 후 60초 뒤 종료
완료 기준	터널 창 없이 접속 → 추출 → 종료. 비밀번호 프롬프트는 앱 다이얼로그로

8.3 메타데이터 브라우저 (데이터는 보여주지 않음)

항목	명세
트리	접속 → 데이터베이스 → 스키마 → 테이블/뷰. 펼칠 때 lazy 로드, 접속당 캐시
데이터베이스	이름, 용량, 인코딩/collation, 접속 가능 여부
테이블 목록	이름, 종류(table/view/materialized/partitioned), **추정 행 수**, 테이블 용량, 인덱스 용량, 합계. 정렬·필터 가능. 수천 개도 한 번의 조회로
정확한 행 수	행마다 "정확히 세기" 버튼. `count(*)` 를 취소 가능하게 실행하고 결과·소요 시간·시각을 표시
테이블 구조	컬럼 순서, 이름, 타입(길이·정밀도 포함), NULL 허용, 기본값, PK/FK/UNIQUE 표시, 코멘트
인덱스	이름, 컬럼 순서, UNIQUE, 종류(btree/gin/…), partial 조건 `WHERE …`, INCLUDE 컬럼, 용량, 사용 횟수 (`idx_scan`, 지원 DB 만)
DDL	테이블·인덱스의 CREATE 문 보기·복사
쿼리 연계	테이블 우클릭 → "전체 추출 쿼리 만들기" (`SELECT 컬럼목록 FROM …`), "컬럼 목록 복사"
하지 않는 것	데이터 미리보기, 행 편집, DDL 실행
완료 기준	1,000개 테이블 스키마가 2초 안에 표시. 행 수에 "추정" 라벨이 항상 보임

8.4 쿼리

항목	명세
편집기	SQL 하이라이트, 줄 번호, 여러 줄. 자동 완성은 없음
파일 참조	`.sql` 파일을 선택하면 내용을 읽기 전용으로 표시하고 파일 변경을 감지해 다시 읽음. 실행 시점의 내용을 사용
여러 문장	`;` 로 나뉜 문장은 거부. 추출 쿼리는 `SELECT` (또는 CTE로 시작) 하나만. `SET` 은 세션 설정 패널로
EXPLAIN	버튼 하나로 실행 계획 조회. 기본은 EXPLAIN (FORMAT JSON) 이며 ANALYZE 없이 플래너만 돌리므로 수십 ms~수 초에 끝난다. 그 세션에 statement_timeout 15초, lock_timeout 3초를 걸어, 다른 세션의 락 때문에 기다리게 되면 3초 안에 실패하고 Monitor 가 락을 잡은 세션(pid·쿼리·경과)을 보여준다. 2026-10-23 DBeaver 에서 EXPLAIN 이 10분 걸린 것은 플래너가 아니라 락 대기 또는 ANALYZE 실행이었고, 이 설계에서는 둘 다 드러난다. EXPLAIN ANALYZE 는 별도 버튼 + "쿼리를 실제로 전부 실행합니다" 확인 다이얼로그 뒤에만 실행. 결과는 예상 행 수·비용·위험 패턴(6.6) 요약과 원문 트리
히스토리	접속별 최근 실행 50건(쿼리, 설정, 결과 행 수, 소요 시간, 파일)
완료 기준	여러 줄 쿼리에 주석이 섞여 있어도 그대로 실행됨 (한 줄 제약 없음)

8.5 세션 설정과 안전 가드

항목	명세
설정 항목	`work_mem` , `statement_timeout` , `lock_timeout` , 병렬 워커 수(PostgreSQL), fetch 배치 크기. DB 가 지원하지 않는 항목은 숨김
기본값	`work_mem` 은 서버 RAM 추정의 1/64 (4GB → 64MB), `statement_timeout` 60분, `lock_timeout` 5초
상한 경고	`work_mem × 3 > RAM × 10%` 면 빨간 경고와 권장값. 실행은 가능
동시 세션 경고	쿼리에 나온 테이블을 읽는 `active` 세션 목록 표시

항목	명세
실행 계획 경고	6.6 의 패턴. Nested Loop 반복 횟수 × 서브쿼리 비용으로 예상 시간대 표시
완료 기준	2026-10-23 의 설정(4GB 서버, 512MB)을 넣으면 경고가 뜨고 64MB 가 권장됨

8.6 출력 설정

항목	명세
포맷	CSV, Parquet. (TSV 는 CSV 의 구분자 옵션)
CSV 옵션	헤더, 구분자, 인용 규칙(RFC 4180 고정), NULL 표기(빈 문자열/ `NULL` /사용자 지정), BOM, 줄바꿈(LF/CRLF), 인코딩 UTF-8 고정
Parquet 옵션	압축(ZSTD/Snappy/none), 행 그룹 크기
압축	gzip, zstd. 파일명에 확장자 자동
분할	N 행마다 또는 M MB 마다. Excel 프리셋 = 1,000,000행 + BOM
경로	폴더 + 파일명 패턴 `{conn}_{name}_{yyyyMMdd_HHmm}` . 덮어쓰기 확인. 디스크 여유 확인
완료 기준	쉼표·따옴표·줄바꿈이 든 값이 CSV 파서(DuckDB, pandas)에서 행 수 그대로 읽힘

8.7 실행

항목	명세
흐름	Preflight → 경고 확인 → 터널 확인 → 세션 설정 적용 → 스트리밍 → 파일 닫기 → 검증 → 완료
검증	완료 후 파일 크기, 행 수(코어가 센 값), 선택적으로 DuckDB 로 다시 세어 대조
취소	즉시 서버 세션 취소 + 파일 `.partial` 로 보존
동시 실행	기본 1개. 설정으로 최대 3개

항목	명세
실패	에러 코드·메시지·서버 메시지 원문·재시도 버튼. 인코딩·OOM·타임아웃은 원인 설명 문장 추가
완료 기준	1,386만 행 3GB 추출이 psql 의 ±10% 시간 안에, 상주 메모리 100MB 이하로 끝남

8.8 모니터

항목	명세
클라이언트 측	경과 시간, 받은 행 수, 받은 바이트, 쓴 바이트, MB/s, ETA(최근 30초 속도 기준)
서버 측	세션 state, wait_event, 서버가 보낸 행 수(지원 DB), 임시 파일 사용량(PostgreSQL `temp_bytes` 증가분)
단계 표시	행이 0 이면 "준비 중(정렬·해시 빌드)", 흐르기 시작하면 "전송 중", `ClientWrite` 대기가 50% 넘으면 "회선 병목"
주기	클라이언트 값 200ms, 서버 값 3~5초
완료 기준	해시 빌드 2분 동안 "준비 중" 이 보이고, 전송 시작 후 ETA 오차가 ±20% 안

8.9 로그

항목	명세
패널	시각, 레벨, 메시지. 서버 NOTICE/WARNING 포함
파일	`tracing` 으로 일별 파일. 비밀번호·토큰은 마스킹
진단 번들	문제 보고용으로 로그 + 설정(비밀번호 제외) + 드라이버 감지 결과를 zip 으로

8.10 로컬 분석 (선택 모듈)

항목	명세
설치	메뉴 첫 사용 시 DuckDB 다운로드(50MB 안팎)

항목	명세
기능	추출된 CSV/Parquet 를 열어 행 수 확인, 컬럼 통계, 필터 후 다른 파일로 저장, CSV ↔ Parquet 변환, Excel 용 분할
경계	여기서도 결과 그리드는 처음 1,000행까지만. 전체 결과는 파일로

8.11 설정 저장

항목	명세
위치	Windows `%APPDATA%\BulkDB\`, macOS `~/Library/Application Support/BulkDB/`
파일	`connections.toml`, `settings.toml`, `history.sqlite`, `drivers/registry.json` (사용자 덮어쓰기용)
동기화	폴더를 Git 이나 클라우드 드라이브에 둘 수 있도록 비밀번호는 절대 포함하지 않음

9. UI 화면 구성

화면은 왼쪽 접속·메타데이터 트리, 가운데 작업 영역, 오른쪽 모니터 패널의 3단 구성이다. 데이터 그리드는 어디에도 없다.

영역	내용	항상 보임
왼쪽 트리	접속 목록 → DB → 스키마 → 테이블. 테이블 행에 추정 행 수·용량 배지. 접속 행에 터널·연결 상태 점	예
가운데 — 탭 "추출"	쿼리 편집기 또는 .sql 파일 참조 / 세션 설정 / 출력 설정 / 실행·EXPLAIN 버튼	기본 탭
가운데 — 탭 "테이블"	트리에서 고른 테이블의 구조(컬럼) / 인덱스 / DDL / 정확히 세기 버튼	테이블 선택 시
가운데 — 탭 "히스토리"	접속별 최근 실행 50건, 다시 실행 버튼	
오른쪽 모니터	현재 작업의 단계, 경과, 행 수, MB/s, ETA, 서버 상태, 취소 버튼. 작업이 없으면 최근 결과 요약	예
하단 로그	접을 수 있는 로그 패널. 에러 발생 시 자동으로 펼침	접힘

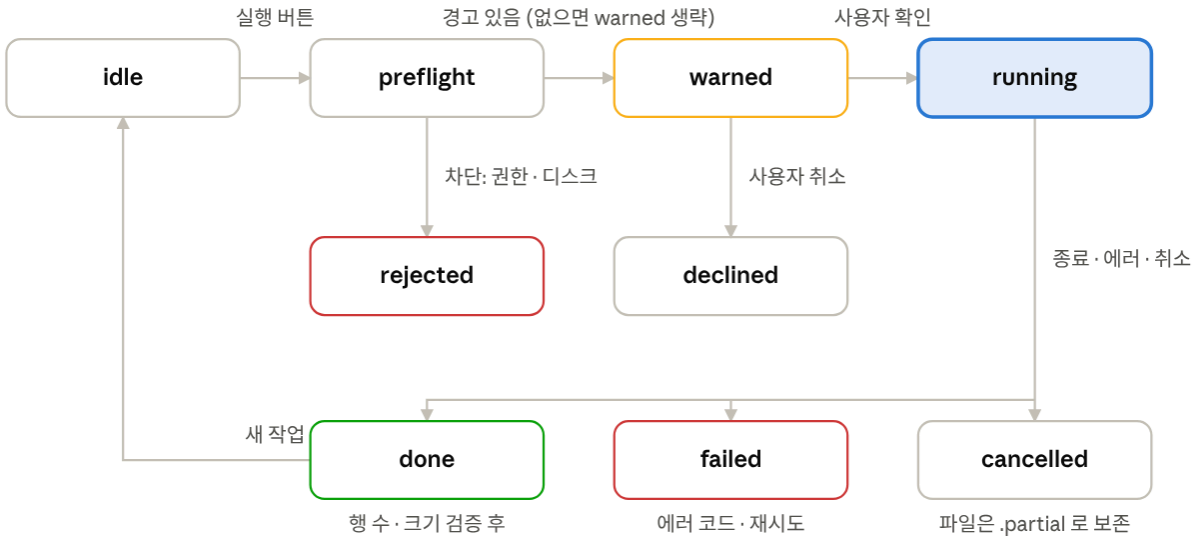
영역	내용	항상 보임
다이얼로그	접속 편집, SSH host key 확인, preflight 경고, 드라이버 설치 안내, 덮어쓰기 확인	

설계 규칙.

- 데스크탑 밀도. 기본 폰트 13px, 행 높이 28px. 웹 앱의 넓은 여백은 쓰지 않는다.
- 상태의 단일 진실은 Rust 다. UI 는 `job:state` 이벤트를 받아 그리기만 하고, 자기 판단으로 상태를 바꾸지 않는다. 앱을 다시 켜도 진행 중인 작업이 모니터에 다시 붙는다.
- 긴 작업 중에도 다른 접속의 메타데이터를 볼 수 있다. 모니터 패널은 작업 목록을 가진다.
- 키보드: Ctrl/Cmd+Enter 실행, Ctrl/Cmd+E EXPLAIN, Esc 취소 확인.

실행은 preflight 를 거쳐 running 으로 가고, 세 가지로 끝난다

작업 하나의 상태 머신 · 상태는 Rust 가 가지고 UI 는 이벤트로 따라간다



추출 작업 상태 머신 · 9 상태

`warned` 는 preflight 가 경고를 낸 상태로, 사용자가 확인해야 `running` 으로 간다. `rejected` 는 경고가 아니라 차단(권한 없음, 디스크 부족)이다. 끝 상태 세 개는 모두 히스토리에 남고, `cancelled` 와 `failed` 는 쓰던 파일을 `.partial` 로 보존한다.

10. Tauri ↔ Next.js 인터페이스

UI 는 `invoke` 로 명령을 보내고 `listen` 으로 이벤트를 받는다. 페이로드 타입은 Rust 에서 `ts-rs` 로 생성하므로 TS 쪽에 손으로 쓴 타입이 없다. 데이터 행은 어떤 경로로도 IPC 를 타지 않는다.

10.1 커맨드 (UI → Rust)

커맨드	인자	반환	비고
`drivers_list`	—	`DriverInfo[]` (id, name, tier, available, installHint)	시작 시 1회 + "다시 감지"
`drivers_rescan`	—	`DriverInfo[]`	
`conn_list` / `conn_save` / `conn_delete`	`Connection`	`Connection[]`	비밀번호는 `conn_save` 인자에만 실리고 keyring 으로 감
`conn_test`	`connId`	`ProbeResult` (version, ramEstimate, defaults)	터널 자동 기동
`catalog_databases` / `catalog_schemas` / `catalog_tables`	`connId`, `db?`, `schema?`	각 Info 배열	접속별 캐시, `refresh: true` 로 갱신
`catalog_columns` / `catalog_indexes` / `catalog_ddl`	`connId`, `TableRef`	`ColumnInfo[]` / `IndexInfo[]` / `string`	
`catalog_exact_count`	`connId`, `TableRef`	`jobId`	취소 가능한 작업으로 돌고 결과는 `job:done` 이벤트
`query_explain`	`connId`, `sql`	`PlanSummary` (rows, cost, warnings[])	
`export_preflight`	`ExportRequest`	`PreflightReport` (warnings[], blockers[], estimates)	실행 전 항상
`export_start`	`ExportRequest`, `acknowledgedWarnings: string[]`	`jobId`	blockers 가 있으면 거부
`job_cancel`	`jobId`	—	
`job_list`	—	`JobSummary[]`	앱 재시작 후 진행 중 작업 복구용
`history_list`	`connId`, `limit`	`HistoryEntry[]`	
`settings_get` / `settings_set`	`Settings`	`Settings`	
`tunnel_confirm_hostkey`	`fingerprint`, `accept`	—	host key 다이얼로그 응답

커맨드	인자	반환	비고
`local_open`	`path`	`jobId`	DuckDB 모듈. 미설치 면 `DRIVER_MISSING` 에러

```
// packages/ipc-types 에 생성되는 형태 (ts-rs). 손으로 수정 금지.
export interface ExportRequest {
  connId: string;
  sql: string;                  // 또는 sqlFile: string
  session: { workMem?: string; statementTimeout?: string; lockTimeout?:
string; fetchBatch?: number };
  output: { dir: string; namePattern: string; format: 'csv' | 'parquet';
            csv?: { header: boolean; delimiter: string; nullAs: string; bom:
boolean; crlf: boolean };
            parquet?: { compression: 'zstd' | 'snappy' | 'none' };
            compress?: 'gzip' | 'zstd';
            split?: { rows?: number; megabytes?: number };
            overwrite: boolean };
}
```

10.2 이벤트 (Rust → UI)

이벤트	페이로드	주기
`job:state`	`{ jobId, state, at }` — 9장의 상태 머신	전이 때마다
`job:progress`	`{ jobId, rows, bytesIn, bytesOut, elapsedMs, rateRowsPerSec, etaSec?, phase }`	**200ms 스로틀**, 값이 안 바뀌면 생략
`job:server`	`{ jobId, state, waitEvent?, serverRows?, tempBytes?, bottleneck?: 'network' | 'server' }`	3~5초
`job:log`	`{ jobId?, level, message, at }`	발생 시
`job:done`	`{ jobId, result: { rows, bytes, files[], durationMs } | { error: { code, message, detail? } } }`	1회
`tunnel:hostkey`	`{ host, fingerprint }`	확인 필요 시
`drivers:changed`	`DriverInfo[]`	재감지 후

10.3 규약

- 진행률은 코어의 원자 카운터를 Tauri 셸이 200ms 마다 읽어 emit 한다. 행마다 emit 하면 IPC 가 병목이 된다.
- 에러는 `{ code, message, detail? }` 하나의 형태로만 온다. `code` 는 `CoreError` 변형 이름 (`CONNECT` , `AUTH` , `QUERY` , `IO` , `CANCELLED` , `TIMEOUT` , `DRIVER_MISSING` , `OOM_RISK`). UI 는 `code` 로 분기하고 `message` 를 그대로 보여준다.
- 커맨드는 모두 `async` 이고 UI 를 막지 않는다. 10초 넘게 걸릴 수 있는 것 (`catalog_exact_count` , `local_open`)은 `jobId` 를 돌려주고 이벤트로 끝을 알린다.
- UI 는 상태를 바꾸지 않는다. 취소 버튼을 눌러도 `job_cancel` 만 보내고, `job:state` 가 `cancelled` 로 올 때까지 "취소 중" 으로 표시한다.
- 비밀번호는 `conn_save` 인자로 한 번 건너간 뒤 UI 메모리에서 지운다. `conn_list` 는 비밀번호 필드를 돌려주지 않는다.
- Tauri 2 capability 파일에 `fs` 는 출력 폴더 선택 다이얼로그와 `.sql` 읽기만, `shell` 은 `ssh` 실행만 허용한다.

11. 크로스플랫폼 (Windows · macOS)

같은 소스에서 Windows x64 설치 파일과 macOS universal(Apple Silicon + Intel) 앱이 나온다. 플랫폼 차이는 `crates/core/src/platform/` 한 모듈에 가두고, CI 가 두 OS 에서 같은 테스트를 돌린다.

11.1 플랫폼별 차이와 처리

항목	Windows	macOS	처리 위치
빌드 툴체인	MSVC (Visual Studio Build Tools), `x86_64-pc-windows-msvc`	Xcode Command Line Tools, `aarch64-apple-darwin` + `x86_64-apple-darwin` → `lipo` universal	CI
WebView	WebView2 (Win10/11 내장, 없으면 bootstrapper 가 설치)	WKWebView (OS 내장)	Tauri 설정
SSH 바이너리	`C:\Windows\System32\OpenSSH\ssh.exe` (Win10 1809+ 내장)	`/usr/bin/ssh` (내장)	`platform::ssh_path()`
SSH 비밀번호 입력	`ssh.exe` 는 콘솔에서만 비밀번호를 받음 → 앱이 pseudo-console(ConPTY) 로 띄우고 프롬프트를 가로챔	`SSH_ASKPASS` + `SSH_ASKPASS_REQUIRE=force` 로 앱 다이얼로그 연결	`tunnel.rs` + `platform`
자격 증명 저장소	Credential Manager (`keyring` → Windows Credential API)	Keychain (`keyring` → Security framework)	`credentials.rs` (크레이트가 추상화)
설정 디렉터리	`%APPDATA%\BulkDB`	`~/Library/Application Support/BulkDB`	`platform::config_dir()`
파일 다이얼로그	Tauri `dialog` 플러그인	동일	UI

항목	Windows	macOS	처리 위치
경로 표기	`\` 구분, 드라이브 문자	`/` 구분	Rust `PathBuf` 로만 다룸. 문자열 조립 금지
줄바꿈 기본	CSV `CRLF` 옵션 기본 켬	`LF` 기본	출력 옵션 기본값만 다름
ODBC	드라이버 매니저 내장 (`odbc32.dll`)	unixODBC 필요 (`brew install unixodbc`), 없으면 ODBC 계열 비활성 + 안내	`drivers/` 감지
Oracle Instant Client	`oci.dll` 탐색	`libclntsh.dylib` 탐색, Apple Silicon 은 arm64 빌드 필요	`registry.json` 의 `requires`
코드 서명	Authenticode (EV 인증서 권장, SmartScreen 평판)	Developer ID 서명 + **notarization** 필수. 안 하면 "손상된 파일" 로 실행 거부	CI 시크릿
설치 파일	NSIS `.exe` (기본) + MSI (조직 배포용)	`.dmg` (universal)	`tauri.conf.json` bundle
자동 업데이트	Tauri updater, 서명된 manifest	동일	공통
창 관리	타이틀바 기본	네이티브 traffic light. 메뉴바에 앱 메뉴 필수 (Cmd+Q 등)	Tauri 메뉴 설정
단축키	Ctrl	Cmd	UI 에서 `platform` 키로 분기

11.2 SSH 비밀번호 인증이 가장 까다로운 지점

키 인증은 두 OS 모두 `ssh -i` 로 끝나지만, **비밀번호 인증**은 `ssh` 가 TTY 에서만 비밀번호를 읽기 때문에 플랫폼별 처리가 필요하다. 2026-10-23 의 운영 환경이 바로 이 경우(root + 비밀번호)다.

- macOS: `SSH_ASKPASS` 환경변수에 앱이 번들한 작은 helper 를 지정하고 `SSH_ASKPASS_REQUIRE=force` 를 주면 `ssh` 가 helper 를 호출해 비밀번호를 받는다. helper 는 앱과 로컬 소켓으로 통신해 다이얼로그 값을 전달한다.
- Windows: Windows OpenSSH 도 `SSH_ASKPASS` 를 지원하지만 버전에 따라 동작이 다르므로, 안정적인 쪽은 **ConPTY** 로 `ssh.exe` 를 띄우고 `password:` 프롬프트를 읽어 입력을 써 넣는 방식이다. `portable-pty` 크레이트를 쓴다.
- 대안으로 순수 Rust SSH(`russh`) 로 터널을 직접 구현하면 두 OS 가 같아지지만, host key·agent·config 파일 호환을 직접 감당해야 한다. v1 은 OS `ssh` 를 쓰고, `russh` 는 v2 검토 항목으로 둔다.

11.3 Apple Silicon

- Rust 와 Tauri 는 universal 빌드를 지원한다. CI 에서 두 타깃을 빌드해 `lipo` 로 합친다.
- 벤더 라이브러리(Oracle, ODBC 드라이버)는 사용자가 **자기 아키텍처에 맞는 것**을 설치해야 한다. 앱이 universal 이어도 arm64 로 실행 중이면 arm64 `.dylib` 만 로드된다. 감지 실패 시 메시지에 아키텍처를 명시한다.
- Rosetta 로 x64 실행은 지원하지 않는다(혼란만 늘어남).

11.4 CI 매트릭스

잡	러너	하는 일
`core-test`	ubuntu (docker 서비스: postgres 15, mysql 8, mssql 2022)	코어 단위·통합 테스트, 벤치
`build-windows`	windows-latest	`cargo clippy` , `tauri build` → NSIS/MSI, Authenticode 서명, smoke test(앱 실행 → 접속 테스트 → SQLite 추출)
`build-macos`	macos-latest (arm64)	universal 빌드, Developer ID 서명, notarization, smoke test
`typegen-check`	ubuntu	`ts-rs` 출력이 커밋된 것과 같은지
`release`	태그 푸시 시	세 산출물 + updater manifest 업로드

smoke test 는 실제 창을 띄우지 않고 `bulk-cli` 로 SQLite → CSV 1만 행을 돌려 두 OS 에서 같은 바이트의 파일이 나오는지(줄바꿈 옵션 제외) 비교한다.

12. 성능·자원 목표와 측정

기준점은 2026-10-23 의 실측이다. 운영 PostgreSQL 15 에서 13,857,273행(CSV 약 3GB)을 SSH 터널(`-C`) 경유 `psql \copy` 로 **126초**에 받았다. 앱은 같은 조건에서 이 수치의 ±10% 안에 들어야 한다.

지표	목표	측정 방법
PostgreSQL CSV 추출 시간	psql `\copy` 의 ±10%	같은 터널·같은 쿼리로 psql 과 앱을 번갈아 3회, 중앙값 비교
상주 메모리 (추출 중)	코어 프로세스 100MB 이하, WebView 포함 300MB 이하	OS 모니터(Task Manager / Activity Monitor) 피크값. 1,386만 행과 1억 행(SQLite 합성)에서 동일해야 함
MySQL / MSSQL 추출 처리율	같은 망 기준 150K 행/s 이상 (9컬럼, 행당 ~200B)	docker 벤치, `bulk-cli --progress` 의 평균 속도
ODBC 추출 처리율	네이티브 경로의 70% 이상	같은 DB 를 네이티브와 ODBC 로 번갈아
CPU	추출 중 코어 1.5개 이하 (네트워크 바운드)	`--progress` 가 함께 찍는 CPU 시간

지표	목표	측정 방법
메타데이터 브라우저	테이블 1,000개 스키마 목록 2초 이내, 컬럼·인덱스 조회 300ms 이내	docker PostgreSQL 에 합성 스키마
앱 시작	콜드 2초 이내 (드라이버 감지 포함)	시작 로그 타임스탬프
설치 파일	Windows 20MB, macOS universal 30MB 이하	CI 산출물 크기 게이트
진행률 ETA 오차	전송 단계 진입 후 ±20%	벤치 로그의 ETA 와 실제 종료 시각 비교

벤치마크 데이터셋. `crates/core/benches/` 에 생성기를 둔다.

- `wide` : 9컬럼 텍스트, 행당 200B, 1,000만 행 — 오늘의 content 추출과 같은 형태
- `narrow` : 3컬럼 정수, 1억 행 — 행 수 기준 메모리 고정 확인
- `nasty` : 쉼표·따옴표·줄바꿈·유니코드·NULL 이 섞인 10만 행 — CSV 정합성
- `cold` : 서버 캐시를 비우고(`pg_prewarm` 반대, 재시작) 측정한 값과 warm 값을 따로 기록

수용 기준. PR 이 코어를 건드리면 CI 가 `wide` 벤치를 돌려 직전 main 대비 처리율 5% 이상 저하 시 실패한다. 메모리 상한은 매 벤치에서 검사한다.

측정 시 주의. 2026-10-23 의 첫 측정(288초)은 콜드 캐시 + `work_mem` 4MB 였고, 두 번째(126초)는 warm + 64MB + `ssh -C` 였다. 조건을 같이 적지 않은 수치는 비교하지 않는다.

13. 보안

이 앱은 운영 DB 의 자격 증명과 SSH 접속 정보를 다룬다. 원칙은 세 가지다. 비밀은 OS 저장소에만, 로그에는 절대 없이, 쿼리는 읽기만.

항목	규칙
비밀번호·키 암호	`keyring` 으로 OS 저장소에만. 설정 파일·히스토리·로그·IPC 응답·크래시 덤프 어디에도 평문 없음. 코드 리뷰 체크리스트 항목
SSH 키 파일	경로만 저장. 파일 권한이 넓으면(Windows: 다른 사용자 읽기, macOS: 0644) 경고
SSH host key	첫 접속 시 지문 확인 다이얼로그. `~/.ssh/known_hosts` 를 그대로 쓴다. `StrictHostKeyChecking=no` 금지
로그 마스킹	`tracing` 레이어에서 `password=` , `PGPASSWORD` , DSN 의 비밀번호 부분, `Authorization` 헤더를 `***` 로. 테스트로 보장

항목	규칙
DB 연결	TLS 는 서버가 지원하면 기본 사용(`sslmode=prefer` 상당). 접속 설정에 `require` / `verify-full` 옵션. 인증서 검증 끄기 옵션은 두지 않는다
쿼리 실행 범위	추출 쿼리는 `SELECT` /CTE 하나만. `INSERT` / `UPDATE` / `DELETE` /DDL 은 파서 단계에서 거부. 세션 설정은 허용 목록(`work_mem` 등)만
읽기 전용 권장	접속 테스트 결과에 계정이 쓰기 권한을 가지면 "읽기 전용 계정 권장" 안내
출력 파일	사용자가 고른 폴더에만 쓴다. 임시 파일(`.partial`)도 같은 폴더. 시스템 temp 사용 안 함
Tauri capability	`fs` : 다이얼로그로 고른 경로 + `.sql` 읽기. `shell` : `ssh` 바이너리만. `http` : updater 엔드포인트만
진단 번들	로그 + 설정(비밀번호 제거) + 드라이버 감지 결과. 만들기 전에 포함 내용을 보여주고 확인
업데이트	Tauri updater 의 서명 검증 필수. 서명 키는 CI 시크릿, 개발자 PC 에 두지 않음
의존성	`cargo audit` , `pnpm audit` 을 CI 게이트로. 드라이버 크레이트는 분기마다 취약점 확인

14. 개발 단계와 마일스톤

순서는 고정이고 기간은 정하지 않는다. 각 phase 는 게이트를 통과해야 다음으로 넘어가고, 특히 Phase 1 의 성능 게이트는 화면을 한 줄도 만들기 전에 통과해야 한다.



개발 로드맵 · 6 phase, 각 phase 끝에 게이트

게이트 중 P0·P1·P4 는 CI 가 자동 검증하고, P2·P3·P5 는 사람이 운영 DB 와 실제 드라이버로 확인한다. Phase 2 의 게이트는 2026-10-23 에 psql 로 한 작업을 앱에서 그대로 재현하는 것이다. 다음 항목은 phase 안에서 병렬로 진행해도 된다.

- Phase 1 안: Parquet Sink 와 압축·분할 Sink 는 CSV 경로가 게이트를 넘은 뒤에 병렬
- Phase 2 안: UI 화면 작업과 Tauri 커맨드 어댑터는 ts-rs 타입만 합의되면 병렬
- Phase 4 안: 세 DB 는 각각 독립. MySQL 부터 시작하고 MSSQL 은 tiberius 의 TLS 설정 확인이 선행 과제

각 phase 가 끝날 때 남길 것: 태그된 릴리스, 게이트 측정 기록(수치·조건·시각), 다음 phase 의 미결 질문 목록.

15. 테스트 전략

코어는 docker 의 실제 DB 로 테스트하고, UI 는 IPC 경계를 mock 해 테스트한다. 성능 벤치는 CI 게이트다.

층	대상	도구	포함되는 것
단위	Sink, 분할, 압축, CSV 인용, 레지스트리 파서, 로그 마스킹, preflight 판정	`cargo test`	`nasty` 데이터셋으로 CSV 왕복(쓰기 → `csv` 크레이트로 읽기 → 행·값 일치)
통합 (DB)	Source·Monitor·Catalog 각 구현	`cargo test --features it` + docker compose (postgres 15, mysql 8, mssql 2022)	1만·100만 행 추출, 취소 중 세션이 실제로 끊기는지, 타임아웃, 인코딩(한글·이모지), NULL, 메타데이터 조회 결과가 `information_schema`와 일치
통합 (터널)	`tunnel.rs`	docker `openssh-server`	키·비밀번호 인증, host key 거부, 포트 충돌, 끊김 후 재연결
성능 벤치	`wide`, `narrow`	`cargo bench` 커스텀 하네스	처리율·피크 메모리 기록, 직전 main 대비 5% 저하 시 실패
IPC 계약	ts-rs 생성 타입	`typegen-check` CI	생성물이 커밋본과 다르면 실패
UI 단위	폼 검증, 상태 표시, 이벤트 → 화면	Vitest + Testing Library	`invoke` / `listen` 을 mock, 상태 머신의 9 상태 각각 렌더
E2E	앱 전체	Tauri WebDriver (`tauri-driver`) on Windows·macOS	SQLite 접속 → 테이블 트리 → 쿼리 → 추출 → 파일 검증. 외부 DB 없이 돌아감
수동 (게이트)	Phase 2·3·5	체크리스트	운영 PostgreSQL 15 에서 1,386만 행 추출, 4GB 서버 512MB 경고, Oracle Instant Client 감지

회귀 데이터셋. `nasty` 는 고정 시드로 생성하고 기대 CSV 를 저장소에 커밋한다. 바이트 단위 비교(줄바꿈 옵션 제외)로 플랫폼 차이를 잡는다.

테스트하지 않는 것. 벤더 ODBC 드라이버의 내부 동작. 드라이버 감지와 연결 실패 메시지까지만 테스트하고, 실제 Oracle 추출은 수동 게이트로 둔다.

16. 리스크와 대응

가장 큰 리스크는 기술이 아니라 범위다. 메타데이터 브라우저가 IDE 로 자라는 것을 막아야 한다.

리스크	영향	대응
범위 확장 — "데이터 몇 행만 보여주자" 가 결과 그리드로 자람	메모리 원칙 붕괴, DBeaver 와 같은 길	1장의 비목표를 PR 템플릿에 넣는다. 데이터 행을 IPC 로 보내는 커맨드는 리뷰에서 거부
Next.js static export 제약 (동적 라우트, 이미지, 서버 기능)	개발 중 뒤늦게 발견	Phase 0 에서 `output: 'export'` 로 빈 앱을 빌드해 둔다. App Router 의 서버 전용 API 는 ESLint 규칙으로 금지
SSH 비밀번호 인증의 플랫폼 차이 (11.2)	Windows 에서 터널이 안 열림	Phase 2 첫 주에 ConPTY 경로를 먼저 검증. 실패 시 `russh` 로 전환
ODBC 드라이버 품질 편차	특정 DB 에서 느리거나 타입 깨짐	7.3 표에 "드라이버 품질에 좌우" 를 명시. 네이티브 4개가 주력, ODBC 는 best effort
macOS 공증 지연·실패	릴리스 막힘	Phase 0 에서 공증 파이프라인까지 CI 에 넣고 매 릴리스 자동화
서버 OOM 재발	운영 장애	preflight 의 work_mem 상한은 차단이 아닌 경고지만, 기본값을 안전하게(RAM/64) 두고 경고 문구에 2026-10-23 사례를 적는다
대용량 CSV 를 Excel 로 열려는 사용자	잘린 파일로 잘못된 판단	Excel 프리셋(100만 행 분할 + BOM)을 출력 설정 첫 줄에 두고, 100만 행 초과 단일 파일 저장 시 안내
PostgreSQL COPY 경로와 Arrow 경로의 결과 불일치 (NULL·인용·날짜 표기)	같은 쿼리가 DB 마다 다른 CSV	`nasty` 왕복 테스트를 모든 Source 에 동일 적용. NULL 표기와 날짜 포맷을 Sink 옵션으로 통일
Tauri 2 / Next 16 메이저 업그레이드	빌드 깨짐	버전 고정(4장), 업그레이드는 전용 PR + 전체 회귀
드라이버 크레이트 유지보수 중단	보안 패치 공백	분기별 의존성 점검, `cargo audit` 게이트

17. 부록 — 설정 파일

Phase 0 에서 그대로 커밋하는 초기 파일들이다. 버전은 4장과 같다.

17.1 루트 `package.json`

```
{
  "name": "bulkdb",
  "private": true,
  "packageManager": "pnpm@12.14.0",
  "engines": { "node": ">=22", "pnpm": "12.14.0" },
  "scripts": {
    "dev": "turbo run dev",
    "build": "turbo run build",
    "lint": "turbo run lint",
    "test": "turbo run test",
    "core:test": "cargo test -p bulk-core",
    "core:bench": "cargo bench -p bulk-core",
    "cli": "cargo run -p bulk-cli --",
    "typegen": "cargo test -p bulk-core --features ts-export export_bindings"
  },
  "devDependencies": { "turbo": "^2" }
}
```

`.npmrc` : `engine-strict=true` 한 줄. pnpm 버전이 다르면 `pnpm install` 이 실패한다.

17.2 `pnpm-workspace.yaml`

```
packages:
  - "apps/*"
  - "packages/*"
  - "crates/*"      # cargo 를 감싼 얇은 package.json 용
```

17.3 `turbo.json`

```
{
  "$schema": "https://turbo.build/schema.json",
  "tasks": {
    "dev":   { "cache": false, "persistent": true },
    "build": { "dependsOn": ["^build"], "outputs": ["out/**", "src-
tauri/target/release/bundle/**"] },
    "lint":  { "dependsOn": ["^build"] },
    "test":  { "dependsOn": ["^build"], "cache": false }
  }
}
```

17.4 apps/desktop/package.json

```
{
  "name": "desktop",
  "private": true,
  "scripts": {
    "dev": "tauri dev",
    "dev:web": "next dev -p 3000",
    "build": "tauri build",
    "build:web": "next build",
    "lint": "eslint . && tsc --noEmit",
    "test": "vitest run"
  },
  "dependencies": {
    "next": "16.4.2",
    "react": "19.3.1",
    "react-dom": "19.3.1",
    "@tauri-apps/api": "^2",
    "@tauri-apps/plugin-dialog": "^2",
    "zustand": "^5",
    "ipc-types": "workspace:*"
  },
  "devDependencies": {
    "typescript": "7.0.2",
    "@tauri-apps/cli": "^2",
    "vitest": "^3",
    "@testing-library/react": "^16",
    "tailwindcss": "^4",
    "eslint": "^9",
    "eslint-config-next": "16.4.2"
  }
}
```

17.5 apps/desktop/next.config.js

```
/** @type {import('next').NextConfig} */
module.exports = {
  output: 'export',              // out/ 에 정적 파일. Tauri 가 이것을 싣는다
  images: { unoptimized: true }, // 이미지 최적화 서버 없음
  trailingSlash: true,
  reactStrictMode: true,
  eslint: { ignoreDuringBuilds: false },
};
```

17.6 `apps/desktop/src-tauri/tauri.conf.json` (핵심 키)

```
{
  "productName": "BulkDB",
  "identifier": "com.example.bulkdb",
  "build": {
    "beforeDevCommand": "pnpm --filter desktop dev:web",
    "devUrl": "http://localhost:3000",
    "beforeBuildCommand": "pnpm --filter desktop build:web",
    "frontendDist": "../out"
  },
  "app": {
    "windows": [{ "title": "BulkDB", "width": 1280, "height": 820,
"minWidth": 1024, "minHeight": 680 }],
    "security": { "csp": "default-src 'self'; style-src 'self' 'unsafe-
inline'" }
  },
  "bundle": {
    "active": true,
    "targets": ["nsis", "msi", "dmg"],
    "windows": { "webviewInstallMode": { "type": "downloadBootstrapper" } },
    "macOS": { "minimumSystemVersion": "12.0" },
    "resources": ["../../../drivers/registry.json"]
  },
  "plugins": {
    "updater": { "endpoints":
["https://updates.example.com/bulkdb/{{target}}/{{arch}}/{{current_version}}"
], "pubkey": "<minisign public key>" }
  }
}
```

17.7 루트 `Cargo.toml`

```
[workspace]
resolver = "2"
members = ["crates/core", "crates/cli", "apps/desktop/src-tauri"]

[workspace.dependencies]
tokio = { version = "1", features = ["full"] }
tokio-postgres = { version = "0.7", features = ["with-chrono-0_4", "with-
uuid-1"] }
mysql_async = "0.34"
tiberius = { version = "0.12", features = ["rustls", "chrono"] }
rusqlite = { version = "0.32", features = ["bundled"] }
odbc-api = "8"
arrow = "53"
```

```
parquet = "53"
csv = "1"
flate2 = "1"
zstd = "0.13"
keyring = "3"
serde = { version = "1", features = ["derive"] }
toml = "0.8"
tracing = "0.1"
thiserror = "2"
ts-rs = "10"
tokio-util = "0.7"   # CancellationToken

[profile.release]
lto = "thin"
codegen-units = 1
strip = true
```

`rust-toolchain.toml`: `[toolchain] channel = "stable"` 에 착수 시점의 정확한 버전(예: `"1.9x.0"`)을 적는다.

17.8 `drivers/registry.json` (발췌)

```
[
  { "id": "postgres", "name": "PostgreSQL", "tier": "builtin", "source":
"postgres",
    "fields": ["host", "port", "database", "user", "password", "sslmode"],
"defaults": { "port": 5432, "sslmode": "prefer" },
    "session_opts": ["work_mem", "statement_timeout", "lock_timeout",
"max_parallel_workers_per_gather"] },
  { "id": "mysql", "name": "MySQL / MariaDB", "tier": "builtin", "source":
"mysql",
    "fields": ["host", "port", "database", "user", "password"], "defaults": {
"port": 3306 },
    "session_opts": ["max_execution_time", "net_read_timeout"] },
  { "id": "mssql", "name": "SQL Server", "tier": "builtin", "source":
"mssql",
    "fields": ["host", "port", "database", "user", "password", "instance",
"encrypt"], "defaults": { "port": 1433, "encrypt": true },
    "session_opts": ["lock_timeout"] },
  { "id": "sqlite", "name": "SQLite", "tier": "builtin", "source": "sqlite",
"fields": ["path"] },
  { "id": "odbc", "name": "ODBC (generic)", "tier": "odbc", "source": "odbc",
    "requires": { "macos": { "library": "libodbc.dylib", "probe":
["/opt/homebrew/lib", "/usr/local/lib"] } },
    "install": { "macos": "brew install unixodbc" },
    "fields": ["driver", "connection_string"] }
]
```

17.9 invoke 호출 예시 (프론트)

```
import { invoke } from '@tauri-apps/api/core';
import { listen } from '@tauri-apps/api/event';
import type { ExportRequest, PreflightReport, JobProgress } from 'ipc-types';

const report = await invoke<PreflightReport>('export_preflight', { req });
if (report.blockers.length) throw new Error(report.blockers[0].message);
const jobId = await invoke<string>('export_start', { req,
acknowledgedWarnings: report.warnings.map(w => w.code) });

const unlisten = await listen<JobProgress>('job:progress', e => {
  if (e.payload.jobId === jobId) store.setProgress(e.payload);   // 200ms 마다
});
```